Hafta 5 — Tag Helper'lar

> **Ders:** Web Programlama · **Hafta:** 5 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** Hafta 4 (form, `name`, Model Binding, PRG) ve Hafta 3 (Razor, `_ViewImports`) **Bu dosya sınav kapsamındadır.** Daha derin konular `H05-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

🎯 Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. Bir Tag Helper'ın **ne olduğunu** ve `@Html` yardımcılarından **farkını** açıklayabileceksiniz.
2. `@addTagHelper` satırının ne işe yaradığını ve **olmadığında ne olduğunu** gösterebileceksiniz.
3. `asp-controller`, `asp-action` ve `asp-route-*` ile **URL üretebilecek**, elle yazılan `href`'in neden kırıldığını **anlatabileceksiniz**.
4. `asp-for`'un ürettiği `name`, `id`, `type` ve `value` değerlerinin **nereden geldiğini** gösterebileceksiniz.
5. `<label asp-for>`, `<textarea asp-for>` ve `<select asp-items>` etiketlerini **kullanabileceksiniz**.
6. `[Display]` ve `[DataType]` attribute'lerinin üretilen HTML'e **nasıl yansıdığını** gösterebileceksiniz.
7. **CSRF** saldırısının nasıl çalıştığını **anlatabileceksiniz**.
8. Tag Helper formunun ürettiği **anti-forgery token**'ı bulabilecek ve `[ValidateAntiForgeryToken]` ile **doğrulatabileceksiniz**.
9. Token'ın neden GET'te değil **POST'ta** gerektiğini açıklayabileceksiniz.
10. Hafta 4'te **sessiz** kalan yazım hatasının bu hafta **derlemede** yakalandığını gösterebileceksiniz.

🧩 Haftanın ilkesi — Çerçeve, doğru olanı kolay yapar

> Tag Helper bir kısaltma değildir. Yazmadığınız `name`'i, yazmadığınız `action`'ı ve **hiç istemediğiniz güvenlik alanını** sizin yerinize üretir. Bir aracın değeri, size yazdırdığı satırlarda değil, **yazmayı unutamayacağınız hâle getirdiği** satırlardadır.

---

1. Tag Helper Nedir?

1.1. Aslında üç haftadır kullanıyorsunuz

Hafta 2'den beri her görünümde şuna benzer satırlar var:

```
<a asp-controller="Kitap" asp-action="Index">Kitaplar</a>
```

Bu bir HTML etiketi **gibi görünüyor**, ama `asp-controller` diye bir HTML attribute'ü **yoktur**. Tarayıcıya giden şey şu:

```
<a href="/Kitap">Kitaplar</a>
```

Arada bir şey oldu: sunucu, etiketi tarayıcıya göndermeden önce **işledi**. O şeyin adı **Tag Helper**.

> Hafta 3'te `ViewData["Title"]` 'ı, Hafta 4'te `HttpContext` 'i böyle tanımıştık: yeni bir şey öğrenmiyoruz, **zaten kullandığımız şeyin adını koyuyoruz.**

Tanıdığınız başka Tag Helper'lar da var:

Zaten kullandığınız	Ne zamandan beri
`<a asp-controller="…" asp-action="…">`	Hafta 2
`<partial name="_KitapKart" model="k" />`	Hafta 3
`<vc:ziyaret-rozeti />`	Hafta 3
`<link … asp-append-version="true" />`	Proje şablonundan beri

Sonuncusu küçük ama öğreticidir:

```
<link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
```

```
<link rel="stylesheet" href="/css/site.css?v=isxqYEJw8LY-Nm8CNlRQ-pmgHVbS5O5BH-rmptf-0vI" />
```

Sonuna eklenen değer dosyanın **içeriğinden** hesaplanır. Dosya değişince değer de değişir; tarayıcı adresi farklı gördüğü için **eski kopyayı kullanmaz**. CSS değiştirdiğinizde "tarayıcı güncellemiyor" sorununun hazır çözümü budur.

1.2. HTML gibi görünen C#

Bir Tag Helper, sunucuda çalışan bir **C# sınıfıdır**. Görevi: bir etiketi yakalamak, attribute'lerine bakmak ve yerine **düz HTML** koymak.

```
Yazdığınız                          Çatı çalışır              Tarayıcıya giden
──────────────────────────────      ──────────────────────    ──────────────────────

<a asp-action="Index">Liste</a>  →  AnchorTagHelper        →  <a href="/Kitap">Liste</a>
<input asp-for="Baslik" />       →  InputTagHelper         →  <input type="text"
                                                                id="Baslik"
                                                                name="Baslik" ... />
```

Buradan **teşhis kuralı** çıkar:

> **Çalışan bir Tag Helper, kendi attribute'ünü siler.** Tarayıcıda (Ctrl+U) `asp-` ile başlayan bir attribute **görüyorsanız**, o Tag Helper **çalışmamıştır**.

Bu kural, bölüm 1.3'teki hatayı saniyeler içinde teşhis etmenizi sağlar.

1.3. `@addTagHelper` — nereden geliyorlar?

Çatı, hangi sınıfların Tag Helper sayılacağını kendiliğinden bilmez. Bunu `Views/_ViewImports.cshtml` dosyasındaki satır söyler:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Okunuşu: "*`Microsoft.AspNetCore.Mvc.TagHelpers` derlemesindeki (`*`) bütün Tag Helper'ları bu klasördeki görünümlerde kullan.*"

Hafta 3'te projenin **kendi** bileşenleri için ikinci bir satır eklemiştik:

```
@addTagHelper *, KampusKitap
```

`<vc:ziyaret-rozeti />` etiketini çalıştıran satır buydu.

🦎 **Satır olmazsa ne olur?** Hata **almazsınız**. Sayfa **200** döner. Ve hiçbir şey çalışmaz:

```
<!-- @addTagHelper olmadan tarayıcıya giden -->
<input asp-for="Metin" />
<a asp-controller="Kitap" asp-action="Index">bağlantı</a>
<form asp-action="Index" method="post">
```

Sonuçlar ağırdır:

Ne olmadı	Sonucu
`<input>` `name` almadı	Hafta 4'ten biliyoruz: `name`'i olmayan alan sunucuya **hiç ulaşmaz**
`<a>` `href` almadı	Bağlantı tıklanabilir bile değil
`<form>` `action` almadı	Form kendi adresine gönderir
Token üretilmedi	Bölüm 4'ün konusu

> 🍼 **`/Etiket/Eksik`** sayfası bunu canlı gösterir. Sayfa `@removeTagHelper` ile Tag Helper'ları bilerek kapatıyor — yani `@addTagHelper` unutulmuş gibi davranıyor.

1.4. `@Html` yardımcılarından farkı

Tag Helper'lardan önce aynı iş `@Html` metotlarıyla yapılıyordu. İkisi de **aynı HTML'i** üretir; fark **yazım biçimindedir**:

```
@* HTML Helper — bir C# metot çağrısı *@
@Html.TextBoxFor(m => m.Baslik, new { @class = "form-control", size = 40 })

@* Tag Helper — bir HTML etiketi *@
<input asp-for="Baslik" class="form-control" size="40" />
```

	`@Html`	Tag Helper
Görünüm	C# metodu	HTML etiketi
HTML attribute eklemek	Anonim nesne: `new { @class = "…" }`	**Doğrudan yazılır**
`class` yazımı	`@class` (çünkü `class` C# anahtar kelimesi)	`class`
Editör yardımı	C# tamamlama	**HTML tamamlama**
Tasarımcı okuyabilir mi?	Zor	**Evet** — dosya HTML'e benziyor

`@Html` **kaldırılmadı**; hâlâ çalışır ve bazı işler yalnızca onunla yapılır (`Html.GetEnumSelectList<T>()` gibi — bölüm 3.4'te kullanacağız). Ama form yazarken tercih edilen yol Tag Helper'dır.

---

2. URL Üreten Tag Helper'lar

2.1. `asp-controller` ve `asp-action`

```
<a asp-controller="Kitap" asp-action="Detay">Detay</a>
```

Çatı bu ikisini alır, **rota tablosuna sorar** ("Kitap/Detay'a hangi adresten gidiliyor?") ve çıkan adresi `href` olarak yazar.

`asp-controller` yazılmazsa **içinde bulunduğunuz controller** varsayılır:

```
@* Etiket controller'ının görünümünde *@
<a asp-action="Index">Ana sayfa</a>      →      <a href="/Etiket">Ana sayfa</a>
```

2.2. `asp-route-*` — yola mı, query string'e mi?

Ek değerler `asp-route-` öneki ile verilir. Nereye yazılacağına **rota şablonu** karar verir:

```
<a asp-controller="Kitap" asp-action="Detay" asp-route-id="3">…</a>
<a asp-controller="Kitap" asp-action="Ara"   asp-route-kelime="suc">…</a>
```

```
<a href="/Kitap/Detay/3">…</a>
<a href="/Kitap/Ara?kelime=suc">…</a>
```

Aradaki farkın sebebi rota şablonudur:

```
pattern: "{controller=Home}/{action=Index}/{id:int?}"
```

- `id` şablonda **var** → değer **yola** yazıldı.
- `kelime` şablonda **yok** → değer **query string'e** düştü.

İkisi birlikte de verilebilir:

```
asp-route-id="3" asp-route-kaynak="menu"   →   /Kitap/Detay/3?kaynak=menu
```

> Hafta 4'te Model Binding'in bu değerleri **okuma** sırasını görmüştük (Form → Rota → Query string). `asp-route-*` aynı tablonun **yazma** yönüdür.

2.3. Elle yazılan `href` neden kırılır?

Bugün iki yazım **aynı** çıktıyı veriyor:

```
<a asp-controller="Kitap" asp-action="Detay" asp-route-id="3">Kitap</a>
<a href="/Kitap/Detay/3">Kitap</a>
```

Her ikisi de `<a href="/Kitap/Detay/3">`. Fark, rota değiştiğinde ortaya çıkar. `Program.cs`'te şablonun başına bir önek ekleyelim:

```
pattern: "site/{controller=Home}/{action=Index}/{id:int?}"
```

Yazım	Rota değişince	Sonuç
`asp-action` ile üretilen	`/site/Kitap/Detay/3` olur	**200**
Elle yazılan `href`	`/Kitap/Detay/3` kalır	**404**

Tuzağın asıl kötü yanı sessiz olmasıdır. Derleyici uyarmaz, uygulama açılır, sayfa görünür. Bağlantının kırık olduğu yalnızca **tıklandığında** anlaşılır — ve genellikle tıklayan siz olmazsınız.

> 💉 `/Etiket/Baglanti` sayfası beş bağlantıyı yan yana basar; `Program.cs`'teki tek satırı değiştirip sayfayı yenilediğinizde dördü taşınır, biri kırılır.

2.4. Form etiketinde aynı mantık

```
<form asp-controller="Kitap" asp-action="EkleTH" method="post">
```

```
<form method="post" action="/Kitap/EkleTH">
```

`action` adresi rotadan üretildi. Ama Form Tag Helper'ı **bir şey daha** yaptı; onu bölüm 4'te göreceğiz.

---

3. Form Tag Helper'ları

Hafta 4'te formu **elle** yazdık:

```
<input type="text" id="baslikAlani" name="Baslik" value="@Model.Baslik" size="40" />
```

Burada **üç** şeyden biz sorumluyduk: `name`'in sınıftaki özellik adıyla aynı olması, `value`'nun modelden gelmesi, `type`'ın alana uygun olması. Üçünde de hata yapabilir ve **uyarı almazdık**.

3.1. `asp-for` — tek attribute, dört çıktı

```
<input asp-for="Baslik" size="40" />
```

```
<input size="40" type="text" data-val="true"
       data-val-required="The Başlık field is required."
       id="Baslik" name="Baslik" value="" />
```

Tek bir `asp-for`, dört ayrı soruyu cevapladı:

Çıktı	Nereden geldi
`name="Baslik"`	Özellik **adından**
`id="Baslik"`	Özellik adından
`type="text"`	Özellik **tipinden** (`string`)
`value=""`	Modeldeki **değerden**

Yazdığımız `size="40"` **olduğu gibi korundu**: Tag Helper tanımadığı attribute'lere dokunmaz.

Kazanç yazım tasarrufu değildir. Asıl kazanç şu:

```
<input asp-for="Balsik" />     @* yazım hatası *@
```

```
error CS1061: 'KitapEkleVM' does not contain a definition for 'Balsik'
```

Proje **derlenmez**. Hafta 4'te aynı hata sayfayı sessizce boş bırakıyordu.

3.2. `type` tipten çıkarılıyor

C# tipi	Üretilen `type`
`string`	`text`
`int`, `int?`	`number`
`decimal`	**`text`** ← aşağıya bakın
`DateTime`	`datetime-local`
`bool`	`checkbox`
`[DataType(DataType.Password)]`	`password` — ve **`value` yazılmaz**
`[DataType(DataType.EmailAddress)]`	`email`

`decimal` neden `number` değil? Sunucu Türkçe kültürde çalıştığı için fiyat `120,50` biçiminde — **virgülle** — yazılır. HTML'in `type="number"` alanı ise yalnızca **noktayı** kabul eder; virgüllü değeri gösteremez. Çatı, kullanıcının yazdığını kaybetmemek için `text` tercih eder. Bu, Hafta 4'te gördüğümüz ondalık ayracı sorununun (`H04-detay.md` bölüm 5) HTML tarafındaki yüzüdür.

`bool` iki alan üretir. Onay kutusunun yanına **gizli** bir alan da konur:

```
<input type="checkbox" id="Onayli" name="Onayli" value="true" />
...
<input name="Onayli" type="hidden" value="false" />
```

Hafta 4'te "işaretlenmemiş onay kutusu gövdede **hiç gelmez**" demiş ve bunu bir Hata Köşesi satırı olarak bırakmıştık. Tag Helper açığı böyle kapatıyor: kutu işaretliyse `true`, değilse gizli alandan gelen `false` gönderilir.

3.3. `<label asp-for>` ve `[Display]`

```
<label asp-for="Baslik"></label>
```

```
<label for="Baslik">Başlık</label>
```

İki şey oldu: `for` değeri alanın `id`'siyle **eşleştirildi** (etikete tıklayınca alan odaklanır), metin ise **modelden** geldi.

Metni belirleyen şey `[Display]` attribute'üdür:

```
[Display(Name = "Fiyat (TL)")]
public decimal Fiyat { get; set; }
```

```
<label for="Fiyat">Fiyat (TL)</label>
```

`[Display]` **bir doğrulama attribute'ü değildir**; yalnızca gösterilecek adı belirler. Yazılmazsa özellik adının kendisi kullanılır.

3.4. `<select asp-items>` ve `SelectList`

```
<select asp-for="KategoriId"
        asp-items="@(new SelectList(KitapDeposu.Kategoriler, "Key", "Value"))">
    <option value="">— seçiniz —</option>
</select>
```

```
<select id="KategoriId" name="KategoriId">
    <option value="">— seçiniz —</option>
    <option value="1">Edebiyat</option>
    <option value="2">Bilgisayar</option>
    <option value="3">Bilim kurgu</option>
</select>
```

`SelectList`'in üç parçası vardır: **kaynak liste**, `value` olacak alanın adı, gösterilecek metnin alanı.

Üç ayrıntı:

1. **Elle yazdığımız `<option>` silinmez.** Tag Helper ürettiklerini onun **arkasına** ekler. "Hiçbiri" seçeneğini isteyen biziz; çatı onu kendiliğinden koymaz.
2. **Seçili olan kendiliğinden işaretlenir.** Modeldeki `KategoriId` 2 ise, çatı o seçeneğe `selected="selected"` yazar. Hafta 4'te bunu elle yapmak gerekiyordu.
3. `enum` **için hazır yardımcı var:** `asp-items="Html.GetEnumSelectList<KategoriTuru>()"`.

3.5. `<textarea asp-for>` ve iç içe nesneler

`<textarea asp-for="Aciklama"></textarea>` aynı mantıkla `name`, `id` ve içeriği üretir.

İç içe bir nesnenin özelliği için:

```
<input asp-for="Adres.Sehir" />
```

```
<input type="text" id="Adres_Sehir" name="Adres.Sehir" value="" />
```

`name` **noktalı,** `id` **alt çizgili.** Sebebi: Model Binding iç içe nesneyi `Adres.Sehir` biçiminde bekler (Hafta 4 bölüm 3.4), ama HTML `id` değerinde nokta CSS ve JavaScript tarafında sorun çıkarır. Çatı ikisini birden doğru yapar.

3.6. `data-val-*` nereden çıktı?

Üretilen HTML'de hiç yazmadığımız attribute'ler var:

```
data-val="true" data-val-required="The Başlık field is required."
```

`[StringLength(50, MinimumLength = 3)]` eklerseniz bunlara `data-val-length-max="50"`, `data-val-length-min="3"` ve ayrıca gerçek bir HTML attribute'ü olan `maxlength="50"` eklenir.

Bu hafta bunları okuyan hiçbir şey yok. Sayfada doğrulama betiği yüklü değil; attribute'ler orada duruyor ve hiçbir işe yaramıyor. Onları okuyan betik ve —asıl önemlisi— **sunucu tarafı doğrulama Hafta 6'nın konusudur.**

Bir ayrıntı bugünden anlaşılmalı: `string` özelliği `data-val-required` aldı, `string?` **almadı**. Bunu yapan bir attribute değil, C#'ın **nullable reference types** özelliğidir: `string` "boş olamaz" demektir.

> 🧪 `/Etiket` sayfası her tip için üretilen HTML'i yan yana basar. Tabloyu elle yazmadık: sayfadaki küçük betik her denetimin kendi `outerHTML`'ini okuyup altına koyuyor.

---

4. CSRF ve Tag Helper'ın Sessiz Katkısı

4.1. Saldırı nasıl çalışıyor?

Diyelim ki kullanıcı sitenizde oturum açtı; tarayıcısında bir oturum cookie'si var. Sonra başka bir sayfaya gidiyor. O sayfada görünmeyen bir form var:

```
<!-- kotuniyet.example sitesindeki sayfa -->
<form method="post" action="https://sizinsite.example/Hesap/ParaGonder">
    <input type="hidden" name="alici" value="saldirgan" />
    <input type="hidden" name="tutar" value="5000" />
</form>
<script>document.forms[0].submit();</script>
```

Kullanıcı hiçbir şey yapmadan form gönderilir. Ve **tarayıcı, sizin sitenizin cookie'sini bu isteğe otomatik ekler** — çünkü adres sizin sitenize gidiyor. Sunucu açısından istek **meşru** görünür.

Bu saldırının adı **CSRF** (Cross-Site Request Forgery).

🧩 Hafta 4'ün ilkesi burada bir adım ilerliyor: istemciden gelen **veri** güvenilir değildi; artık isteğin **kaynağı** da güvenilir değil.

4.2. Token nereden geliyor?

Savunma şudur: sunucu forma, **kendi ürettiği** gizli bir değer koyar ve POST'ta onu arar. Saldırganın sayfası bu değeri **bilemez**, çünkü başka bir sitenin sayfasının içeriğini okuyamaz.

Tag Helper bunu **siz istemeden** yapar:

```
<form asp-controller="Kitap" asp-action="EkleTH" method="post">
```

```
<form method="post" action="/Kitap/EkleTH">
    ...
    <input name="__RequestVerificationToken" type="hidden" value="CfDJ8A4-mf96z…" />
</form>
```

Aynı formu **elle** yazarsanız:

```
<form method="post" action="/Kitap/EkleTH">
```

```
<form method="post" action="/Kitap/EkleTH">
    ...
</form>
```

Token yok. Hafta 4'te yazdığımız bütün formlar bu durumdaydı.

4.3. `[ValidateAntiForgeryToken]` — üretmek yetmez

Token'ın **üretilmesi** tek başına hiçbir şey yapmaz; sunucudan onu **kontrol etmesini** istemek gerekir:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult EkleTH(KitapEkleVM girdi) { … }
```

Üç durumu ölçelim:

İstek	Sonuç
Tag Helper formundan (token var, cookie eşleşiyor)	**302** — action çalıştı, PRG devrede
Token **yok**	**400 Bad Request**
Token var ama **cookie yok**	**400 Bad Request**

Reddedilen istekte dikkat edilecek üç şey var:

- Ekranda **hiçbir açıklama yok** — boş bir sayfa gelir.
- Bir **exception değildir**; geliştirici hata sayfası bile görmezsiniz.

- Action'ın içindeki kod **hiç çalışmamıştır**; istek oraya ulaşmadan reddedilir.

> **Son satır neden önemli?** Çünkü bu, doğru davranıştır. Kod çalışıp sonra "acaba meşru mu?" diye sormaz; istek **kapıdan içeri alınmaz**.

Her POST action'ına tek tek yazmak yerine uygulamanın tamamına uygulanabilir: `[AutoValidateAntiforgeryToken]`. Ayrıntısı `H05-detay.md` bölüm 3'tedir.

4.4. Token neden GET'te yok?

Aynı Tag Helper, `method="get"` ile yazıldığında **token üretmez**:

```
<form asp-action="Ara" method="get">   →   <form method="get" action="/Kitap/Ara">
```

Sebebi Hafta 4'ün kuralıdır: **okuma GET, değiştirme POST**. CSRF, kullanıcıya istemediği bir **değişikliği** yaptırma saldırısıdır. Veri değiştirmeyen bir GET isteğinin sahtesini göndermek saldırgana bir şey kazandırmaz.

> Bunun tersi de doğrudur: veri **değiştiren** bir işlemi GET'e koyarsanız, token koruması da devre dışı kalır. "Silme bağlantısı" olarak yazılan `<a href="/Kitap/Sil/3">` bu yüzden tehlikelidir.

4.5. Tek başına cookie neden yetmez?

Saldırganın sayfası ne yapabilir?

Yapabilir mi?	
Sizin sitenize form göndermek	**Evet** — HTML'i kendi sayfasına yazar
Tarayıcının cookie'sini ekletmek	**Evet** — tarayıcı bunu otomatik yapar
Token'ın değerini öğrenmek	**Hayır** — başka sitenin sayfasını okuyamaz

Koruma tam olarak bu boşluktan doğar: **cookie otomatik gider, gizli alan gitmez.** Sunucu ikisinin **eşleştiğini** arar. Bu yüzden yalnızca cookie göndermek yetmez — ölçtük, **400** aldık.

> 🍼 `/Etiket/Csrf` sayfasında üç form yan yana duruyor: Tag Helper + POST, Tag Helper + GET, elle yazılmış POST. İkisi çalışır, biri 400 döner.

🐛 Hata Köşesi — Hafta 5

Belirti	**Gerçek sebep**	**Çözüm**
Sayfada `asp-for="…"` **yazısı görünüyor**	Tag Helper işlenmedi	`_ViewImports.cshtml`'de `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers` satırı var mı?

`error CS1963: An expression tree may not contain a dynamic operation`	`asp-for` kullanılan görünümde `@model` **yok**	Görünümün başına `@model` yönergesini ekleyin
`error CS1061: … does not contain a definition for '…'`	`asp-for` değeri yanlış yazılmış	Özellik adını düzeltin — **bu hafta derleyici yakalıyor**
Form gönderiliyor ama **400** dönüyor	`[ValidateAntiForgeryToken]` var, form **elle** yazılmış	Formu `<form asp-action="…" method="post">` yapın
Token üretiliyor ama **hiç kontrol edilmiyor**	Action'da `[ValidateAntiForgeryToken]` yok	Attribute'ü ekleyin ya da `[AutoValidateAntiforgeryToken]` kullanın
Formda token **yok**	Form `method="get"`	GET'te token üretilmez; değiştiren işlem POST olmalı
Bağlantı **404** veriyor, dün çalışıyordu	`href` elle yazılmış, rota değişmiş	`asp-controller` / `asp-action` kullanın
`<select>` doğru seçeneği **işaretlemiyor**	`asp-for` yok, yalnızca `asp-items` var	`asp-for` ile modeldeki özelliği bağlayın
Onay kutusu hâlâ **false** geliyor	Form Tag Helper'ı devrede değil → gizli alan üretilmedi	`<input asp-for="…" />` kullanın
`type="number"` beklerken `text` geldi	Özellik `decimal`	Normaldir — ondalık ayracı yüzünden (bölüm 3.2)

Klasik Tuzak — Adresi elle yazmak

```
<form action="/Kitap/Ekle" method="post">        @* ❌ *@
<a href="/Kitap/Detay/3">Detay</a>               @* ❌ *@
```

Bugün çalışır. İki şeyi birden kaybedersiniz:

1. **Rota değişince kırılır** — sessizce, 404 olarak.
2. **Anti-forgery token üretilmez** — form Tag Helper'ı devreye girmediği için.

İkinci madde daha sinsidir: POST action'ınıza sonradan `[ValidateAntiForgeryToken]` eklediğinizde form **400** vermeye başlar ve sebebi uzaktan görünmez.

```
<form asp-controller="Kitap" asp-action="Ekle" method="post">   @* ✅ *@
<a asp-controller="Kitap" asp-action="Detay" asp-route-id="3">Detay</a>   @* ✅ *@
```

📖 Terimler Sözlüğü

Terim	Ne demek
Tag Helper	Bir HTML etiketini sunucuda işleyip düz HTML üreten C# sınıfı
`@addTagHelper`	Hangi derlemedeki Tag Helper'ların kullanılacağını bildiren directive
`asp-for`	Bir alanı model özelliğine bağlayan attribute; `name`, `id`, `type`, `value` üretir
`asp-route-*`	URL'ye ek değer ekler; rota şablonuna göre yola ya da query string'e yazılır
`asp-items`	`<select>` seçeneklerinin kaynağı
`SelectList`	Kaynak listeyi `value`/metin çiftlerine dönüştüren sınıf
`[Display]`	Gösterilecek adı belirler — **doğrulama attribute'ü değildir**
`[DataType]`	Alanın anlamını belirtir; `type` çıkarımını etkiler
CSRF	Kullanıcının oturumunu kullanarak onun adına istek gönderten saldırı
Anti-forgery token	Sunucunun forma koyduğu, saldırganın okuyamayacağı gizli değer
`__RequestVerificationToken`	O token'ı taşıyan gizli alanın adı
`[ValidateAntiForgeryToken]`	Action'a gelen isteğin token'ını doğrulatan attribute

✅ Kontrol Soruları

A. Tag Helper nedir

1. Tarayıcıya giden kaynakta `asp-for="Baslik"` yazısını görüyorsunuz. Ne ters gitti?
2. `@addTagHelper` satırı silinirse sayfa hangi durum kodunu döner? Bu neden tehlikelidir?
3. `@Html.TextBoxFor(...)` ile `<input asp-for="...">` arasındaki fark üretilen HTML'de mi, yazımda mı?
4. `@Html` yardımcılarına hâlâ ihtiyaç duyduğumuz bir yer söyleyin.

B. URL üretimi

5. `asp-route-id="3"` neden yola, `asp-route-kelime="suc"` neden query string'e yazıldı?
6. `asp-controller` yazılmazsa çatı hangi controller'ı varsayar?
7. Rota şablonunun başına `site/` eklendi. Elle yazılmış `href="/Kitap/Detay/3"` ne döner? `asp-action` ile üretilen?
8. Elle adres yazmanın anti-forgery ile ne ilgisi var?

C. Form Tag Helper'ları

9. `<input asp-for="Fiyat" />` niçin `type="number"` değil `type="text"` üretiyor?
10. `bool` bir özellik için kaç `<input>` üretilir? İkincisi ne işe yarar?
11. `asp-for="Adres.Sehir"` hangi `name` ve hangi `id` değerini üretir? Neden farklılar?
12. `[Display(Name = "…")]` bir doğrulama attribute'ü müdür? Ne yapar?

13. `<select>` içine elle yazdığınız `<option>` silinir mi?
14. `data-val-*` attribute'lerini bu hafta ne okuyor?
15. `string` ile `string?` arasındaki üretim farkını hangi C# özelliği doğuruyor?

D. CSRF

16. Saldırganın sayfası cookie'nizi gönderebiliyorsa, token'ı neden gönderemiyor?
17. Token'ı üretmek tek başına yeterli mi? Değilse eksik olan ne?
18. Token'sız POST hangi durum kodunu alır? Ekranda ne görünür?
19. Aynı formu `method="get"` yaparsanız token üretilir mi? Neden?
20. "Silme bağlantısı" olarak `<a href="/Kitap/Sil/3">` yazmak neden tehlikelidir?

📑 Kaynakça

> Her kaynağın yanında **bu ders notunun hangi bölümünü** dayandırdığı yazılıdır. Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**: sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür. **Örnekte `Startup.cs` görüyorsanız yanlış sürümdesiniz.**

Zorunlu okuma

Kaynak	Dayandırdığı bölüm
Tag Helpers in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/intro?view=aspnetcore-10.0	Bölüm 1 tamamı — Tag Helper nedir, `@addTagHelper`, `@Html` ile karşılaştırma
Tag Helpers in forms in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0	Bölüm 3 tamamı — `asp-for`, `label`, `textarea`, `select`
The Select Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0#the-select-tag-helper	Bölüm 3.4 — `asp-items`, `SelectList`, seçili öğe
Anchor Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/anchor-tag-helper?view=aspnetcore-10.0	Bölüm 2 — `asp-controller`, `asp-action`, `asp-route-*`

Güvenlik

Kaynak	Dayandırdığı bölüm
Prevent XSRF/CSRF attacks in ASP.NET Core — https://learn.microsoft.com/aspnet/core/security/anti-request-forgery?view=aspnetcore-10.0	Bölüm 4 tamamı — token üretimi, doğrulama, `[AutoValidateAntiforgeryToken]`
Cross-Site Request Forgery (CSRF) (MDN) — https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/CSRF	Bölüm 4.1 — saldırının tarayıcı tarafındaki tanımı

Köprüler (sonraki haftalara)

Kaynak	Dayandırdığı bölüm
Model validation in ASP.NET Core MVC — https://learn.microsoft.com/aspnet/core/mvc/models/validation?view=aspnetcore-10.0	Bölüm 3.6 — `data-val-*` attribute'lerini kim okuyor (**Hafta 6**)
Author Tag Helpers — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/authoring?view=aspnetcore-10.0	Kendi Tag Helper'ınızı yazmak (`H05-detay.md` bölüm 1)

📚 Daha Fazlası

`H05-detay.md` dosyasında (**sınav kapsamı dışı**):

- Kendi Tag Helper'ınızı yazmak: `TagHelper` sınıfı, `Process`, `[HtmlTargetElement]`
- `asp-append-version` ve içerik parmak izi (cache busting)
- `<environment>`, `<partial>`, `<cache>` Tag Helper'ları
- `[AutoValidateAntiforgeryToken]` ve token'ı AJAX isteklerinde taşımak
- `SameSite` cookie attribute'ünün CSRF'e katkısı (Hafta 7 köprüsü)
- API'lerde CSRF neden farklı ele alınır (Hafta 12 köprüsü)
- Doğrulama mesajlarının Türkçeleştirilmesi

➡️ Haftaya Hazırlık

Bu hafta `data-val-required`, `data-val-length-max` gibi attribute'lerin HTML'e **kendiliğinden** yazıldığını gördük — ve onları **okuyan hiçbir şey olmadığını**.

Hafta 6'da o attribute'leri okuyan tarayıcı tarafını ve asıl önemlisi **sunucu tarafı doğrulamayı** kuracağız.

Düşünerek gelin: Tarayıcıdaki doğrulama kullanıcıya yardım eder, hatayı erken gösterir. Peki kullanıcı tarayıcının doğrulamasını **atlatırsa** ne olur? Hafta 4'ün ilkesi — *istemciden gelen hiçbir veri güvenilir değildir* — burada hâlâ geçerli mi?

> 🏠 Ev göreviniz bu sorunun zeminini hazırlıyor: Hafta 4'teki formunuzu Tag Helper'larla yeniden yazıp iki HTML çıktısını **yan yana** koyacaksınız. `data-val-*` satırlarını da işaretleyin — haftaya onlarla başlayacağız.